Large-Scale Terrain Characterisation: Slope and Aspect Algorithms, Their Computational Cost, and Multi-Resolution Visualisation of DEMs

Course: Spatial Informatics
Team name: GeoGuessers
Siddhant Gudwani (2024102042), Swayam Goyal (2024111012)
Siddharth Choudhary (2024111021), Yakkala Viswanadha Kartikeya (2024111033)

1. Introduction

Digital elevation models (DEMs) underpin most quantitative descriptions of terrain, from them, GIS tools such as GDAL and QGIS derive local attributes (slope, aspect, hillshade, curvature, roughness, TRI and TPI) and non-local ones such as viewsheds and least-cost terrain distances. computing these for a small grid is easy. Computing them over a Himalayan DEM at resolutions from 30 m down to 1 m, and displaying the result fast enough to pan and zoom on a laptop is a different problem: at 1 m, a grid covering a few hundred kilometres holds tens of billions of cells far more then such a machine can keep in memory. This review examine that gap. sixteen studies at global, continental and local scales are grouped into four theme's: local terrain operators, resolution and data volume, visibility and distance algorithms, and the scaling of computation and display. For each theme we ask which method is the naive baseline, which is the state of the art and whether computational cost is reported.

2. Review of Existing Studies

Local terrain operators. Slope, aspect, hillshade and curvature, are all computed from a 3 $\times$ 3 window. Horn (1981) weights the cardinal neighbours twice as heavily as the diagonals while Zevenbergen and Thorne (1987) fit a partial quartic through all nine cells from which curvature also follows. GDAL uses Horns method by default and offer's Zevenbergen and Thorne's as an option. guth and Kane (2021) adapted slope, aspect and hillshade to non-square cells, the normal case for DEMs in geographic coordinates and found that computed slope changes roughly logarithmically with grid spacing. Trevisani, Teza and Guth (2023) revisited the topographic ruggedness index (TRI), and proposed a radial roughness index to improve it. TPI, the difference between a cell and the mean of its neighbourhood uses the same logic with a larger window, so its cost grows with window size. every operator in this group visits each cell once and costs $O(N)$ so a naive and a state-of-the-art version share the same complexity, and differ only in how they are implemented.

Resolution and data volume. Deng, Wilson and Bauer (2007) computed six attributes at several resolutions at fixed points, and found that resolution changes what slope measures at a

point by an amount that depends on it's position in the landform. Polidori and El Hage (2020) argue that source error propagates into every derivative so nominal resolution is a weak guide to a DEM's usefulness. the global products show the cost side. Amatulli et al. (2020) computed geomorphometric layers including roughness, TRI, TPI and curvature, for the whole globe and Ho et al. (2025) note that a 1 arc-second global grid hold about 202 billion pixels, which could only be processed by tiling with overlap running tiles in parallel and mosaicking. Both precompute every layer at fixed scales. For a benchmark this mean run time and accuracy must be measured together, since a coarser grid is both faster, and different.

Visibility and distance. these operations are not local so here the naive, and state of the art methods differ in complexity. The exact R3 viewshed traces a separate line of sight to every cell, and costs $O(N^{1.5})$ whereas XDraw approximates visibility ring by ring from the observer at $O(N)$ (Sanchez-Fernandez et al., 2021). Cauchi-Saunders and Lewis (2015) mapped XDraw to the GPU, and outperformed CPU and GPU versions of R3 R2 and XDraw and Dou, Li and Wang (2019) split parallel XDraw into equal-area triangles to balance the load. Sanchez-Fernandez et al. (2021) then showed that restructuring the DEM in memory, mattered more than adding GPU's, reaching up to 827 times the throughput of the previous total-viewshed algorithm. for terrain distance, Hart, Nilsson and Raphael (1968) showed that A* with an admissible heuristic still find's the optimal path, while expanding fewer nodes than an uninformed Dijkstra-style search.

Scaling computation and display. Barnes (2016) tiled the DEM, and filled depressions through a producer-consumer scheme that runs unchanged on one core, many cores or a cluster reaching trillion-cell datasets on ordinary hardware. Yu, Zhou and Li (2024) scheduled neighbourhood operations on the CPU and GPU together, and reached a speedup of about 29 on a 1.7 GB raster a fifth better than their GPU-only version. the gain came from giving each processor the part of the work it handles best so using both, beat using the GPU alone. For display, Losasso and Hoppe (2004) cache terrain in nested grids centred on the viewer so detail follows the view: fine near the camera, and coarse further away with a steady frame rate regardless of dataset size and Campos et al. (2020) built a multi-resolution pyramid for virtual globes one tile at a time, because a continent-scale mesh would not fit in commodity memory and displayed it in Cesium, a browser-based globe. these pyramids carry elevation only, derived attributes are still computed in advance.

Table 1. our preliminary test — GDAL slope run times on our laptop, (method in appendix B).

DEM	Grid (cells)	Algorithm	Runs (s)	Median (s)
30 m	≈ 12.1 M	Horn	3.76 (first, uncached), 0.51, 0.49	0.51
30 m	≈ 12.1 M	ZT	0.45, 0.44, 0.58	0.45
15 m	48.6 M	Horn	1.71, 1.42, 1.86	1.71
15 m	48.6 M	ZT	1.91, 1.91, 1.99	1.91
10 m	109.3 M	Horn	5.67, 5.39, 4.98	5.39
10 m	109.3 M	ZT	5.06, 4.69, 4.85	4.85
5 m	437.1 M	ZT	19.62, 18.72, 20.00 (uncached); 5.96 (cached)	19.62 / 5.96
5 m	437.1 M	Horn	20.49 (uncached); 8.23, 7.44 (cached)	20.49 / 7.84

3. Critical Discussion and Research Gap

studies agree that local operators differ more in resolution sensitivity, than in cost that tiling with overlap is the accepted way to handle grids larger than memory and that data movement usually limits performance more than arithmetic. Our pilot test agrees at 5 m, file caching changed GDALs slope run time about threefold far more than the choice of slope operator (Table 1). For slope, aspect, TRI or TPI naive and state-of-the-art versions share $O(N)$ complexity so the comparison is between implementations such as GDAL against a tiled or GPU version. Only visibility and distance offer a true algorithmic contrast: R3 against XDraw and Dijkstra against A*. Reported speedups range from 29 to over 800 times across differing baselines and hardware and no study separates disk read, CPU to GPU transfer compute and rendering, so when a GPU helps is unclear. three gaps follow. No study report's where standard GDAL or QGIS terrain tools stop being practical, on a single laptop. No study uses pyramids to compute derivatives on demand because slope depends on cell size, nobody has measured the error of coarse-level slope against the time it saves. future studies should also measure XDraws accuracy loss in high-relief terrain.

4. Conclusion

the literature agrees that slope, aspect, hillshade, curvature, TRI and TPI are stable local operators standardised in tools such as GDAL and that resolution affects their output, more than the choice of operator. The dominant responses to scale are tiling with overlap CPU and GPU parallelism with attention to memory layout and pyramid-based display. Non-local operations differ: XDraw and A* are the faster, approximate or heuristic alternatives to R3 and exhaustive search. What is missing, is a stage-by-stage benchmark of these standard tools on commodity hardware as DEM size grows and an evaluation of computing derivatives on demand at the

viewed pyramid level. this project address's both with GDAL as the baseline, and QGIS and Cesium for display.

5. References

Amatulli, G., McInerney, D., Sethi, T., Strobl, P., & Domisch, S. (2020). Geomorpho90m, empirical evaluation and accuracy assessment of global high-resolution geomorphometric layers. Scientific Data, 7, 162.

Barnes, R. (2016). Parallel Priority-Flood depression filling for trillion cell digital elevation models on desktops or clusters. Computers & Geosciences, 96, 56–68.

Campos, R., Quintana, J., Garcia, R., Schmitt, T., Spoelstra, G., & Schaap, D. M. A. (2020). 3D simplification methods and large scale terrain tiling. Remote Sensing, 12(3), 437.

Cauchi-Saunders, A. J., & Lewis, I. J. (2015). GPU enabled XDraw viewshed analysis. Journal of Parallel and Distributed Computing, 84, 87–93.

Deng, Y., Wilson, J. P., & Bauer, B. O. (2007). DEM resolution dependencies of terrain attributes across a landscape. International Journal of Geographical Information Science, 21(2), 187–213.

Dou, W., Li, Y., & Wang, Y. (2019). An equal-area triangulated partition method for parallel Xdraw viewshed analysis. Concurrency and Computation: Practice and Experience, 31(17), e5216.

Guth, P. L., & Kane, M. (2021). Slope, aspect, and hillshade algorithms for non-square digital elevation models. Transactions in GIS, 25(5), 2309–2332.

Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. IEEE Transactions on Systems Science and Cybernetics, 4(2), 100–107.

Ho, Y.-F., Grohmann, C. H., Lindsay, J., Reuter, H. I., Parente, L., Witjes, M., & Hengl, T. (2025). GEDTM30: Global ensemble digital terrain model at 30 m and derived multiscale terrain variables. PeerJ, 13, e19673.

Horn, B. K. P. (1981). Hill shading and the reflectance map. Proceedings of the IEEE, 69(1), 14–47.

Losasso, F., & Hoppe, H. (2004). Geometry clipmaps: Terrain rendering using nested regular grids. ACM Transactions on Graphics, 23(3), 769–776.

Polidori, L., & El Hage, M. (2020). Digital elevation model quality assessment methods: A critical review. Remote Sensing, 12(21), 3522.

Sanchez-Fernandez, A. J., Romero, L. F., Bandera, G., & Tabik, S. (2021). A data relocation approach for terrain surface analysis on multi-GPU systems: A case study on the total viewshed problem. International Journal of Geographical Information Science, 35(8), 1500–1520.

Trevisani, S., Teza, G., & Guth, P. L. (2023). Hacking the topographic ruggedness index. Geomorphology, 439, 108838.

Yu, Z., Zhou, C., & Li, M. (2024). A parallel strategy to accelerate neighborhood operation for raster data coordinating CPU and GPU. Cartography and Geographic Information Science, 51(6), 813–829.

Zevenbergen, L. W., & Thorne, C. R. (1987). Quantitative analysis of land surface topography. Earth Surface Processes and Landforms, 12(1), 47–56.

Appendix A: Division of Work

Each team member reviewed four papers on a different theme, the team then compared the findings, across all sixteen papers to write Sections 2 to 4.

Appendix B: How We Ran the Preliminary Test

Before the main benchmarking we tested whether the review's central claim, that data movement limits large-DEM processing more than arithmetic holds for the standard GDAL slope tool on our own laptop.

We took a Copernicus GLO-30 tile of the Uttarakhand Himalaya (N30°, E79°), and reprojected it in QGIS to WGS 84 / UTM zone 44N (EPSG:32644) at 30 m with bilinear resampling. Grids of 15 10 and 5 m were derived by nearest-neighbour resampling so they add cells but no terrain detail, and are used only for timing.

we ran QGIS 3.44.12's Slope tool (Raster, Analysis, Slope) which calls gdaldem from GDAL 3.13.1 with Horn's and Zevenbergen and Thorne's (ZT) methods three times each per grid, writing the output as a temporary GeoTIFF. The laptop runs Windows with 24 GB RAM an NVMe SSD and an NVIDIA GeForce RTX 3050 A Laptop GPU (4 GB), which gdaldem does not use. Run times come from the QGIS processing log and the memory of the QGIS process was read from Windows Task Manager.

Run time grew roughly in line with the number of cells: from 30 m to 10 m the cells rose 9 times, and the median time about 10.7 times. Horn and ZT took almost the same time at every size caching mattered far more. At 5 m runs before the 1.7 GB input was cached took 19 to 20 s and repeat runs took 6 to 8 s, about three times faster; the first 30 m run was likewise seven times slower than repeats. QGIS memory stayed between about 1.2 and 1.5 GB, because gdaldem reads the DEM a few rows at a time.

this is a pilot. The finer grids are resampled from 30 m memory was measured for QGIS, and not for the gdaldem process and the 2 m grid (about 2.7 billion cells roughly 11 GB) was not run. These will be covered in the main benchmark.